Laboratorio 4: REST APIs

HTTP, consumo desde Java, protocolos y diseño de una API con OpenAPI

Ingeniería de Software II (2016702)

Daniel Alonso Gracia Pinto

Noviembre de 2026

Grupo 00

Ejercicio 0. Anatomía de una petición HTTP

Las peticiones van a `https://httpbin.org`, que devuelve en el cuerpo lo que recibió: los parámetros, los headers y el cuerpo de la petición.

(a) Partes de la URL

La petición es `GET https://httpbin.org/get?curso=seii&grupo=3`.

Parte	Valor
Esquema	`https`
Host	`httpbin.org`
Ruta	`/get`
Parámetros de consulta	`curso=seii`, `grupo=3`

El signo `?` separa la ruta de los parámetros y `&` separa un parámetro del siguiente. En la respuesta, los parámetros aparecen en el campo `args`, ya convertidos en un objeto: `{"curso": "seii", "grupo": "3"}`. El `3` llega como texto, porque en la URL todo es texto y el servidor no sabe que era un número.

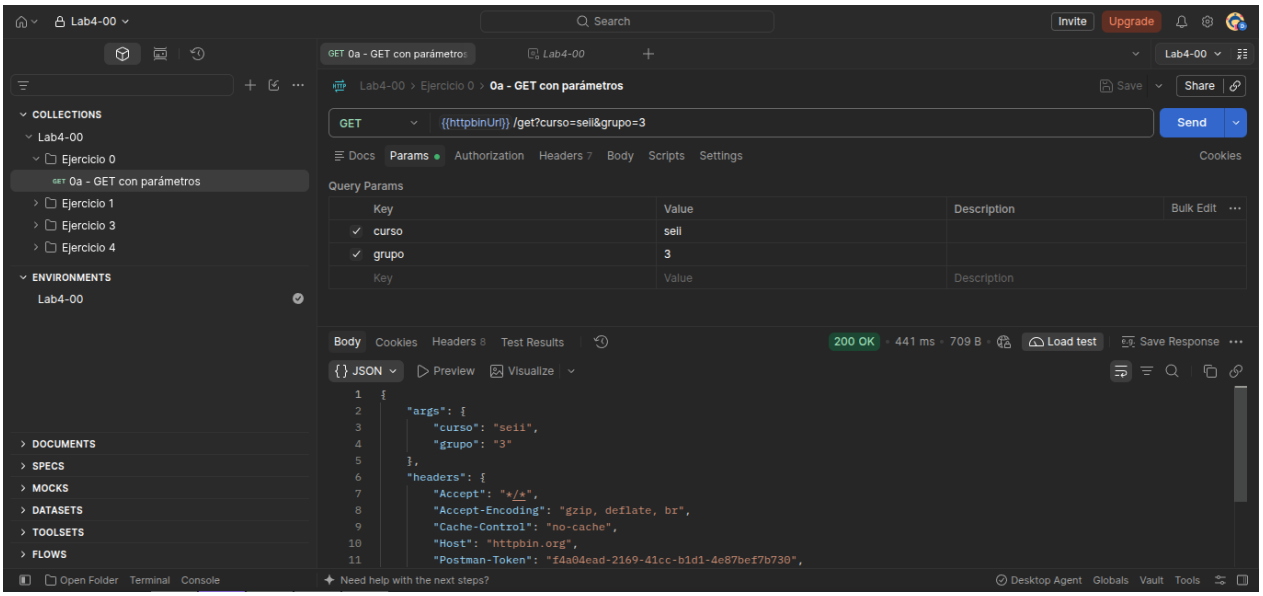


Figura 1. `GET /get` con los parámetros `curso` y `grupo`.

(b) Headers

Además del `X-Equipo: Musubi Japan`, Postman agregó por su cuenta siete headers: `Cache-Control` (`no-cache`), `Postman-Token` (un identificador distinto en cada envío), `Host`, `User-Agent` (`PostmanRuntime/2.12.2`), `Accept` (`*/*`), `Accept-Encoding` (`gzip, deflate, br`) y `Connection` (`keep-alive`). El campo `headers` de la respuesta los muestra todos menos `Connection`: ese header solo vale entre dos equipos vecinos de la red, y el intermediario que tiene httpbin.org adelante no lo reenvía.

`User-Agent`. Identifica el programa que hace la petición. El servidor lo usa para estadísticas, para adaptar la respuesta o para bloquear clientes. Con `curl` la misma petición llega con `curl/8.18.0`.

`Accept`. Dice qué formatos de respuesta acepta el cliente. `*/*` significa cualquiera; una app que solo entiende JSON debería enviar `application/json`.

`Host`. Dice a qué sitio va la petición. Un mismo servidor, con una sola IP, puede atender muchos dominios, y por el `Host` sabe cuál se pidió. En HTTP/1.1 es obligatorio.

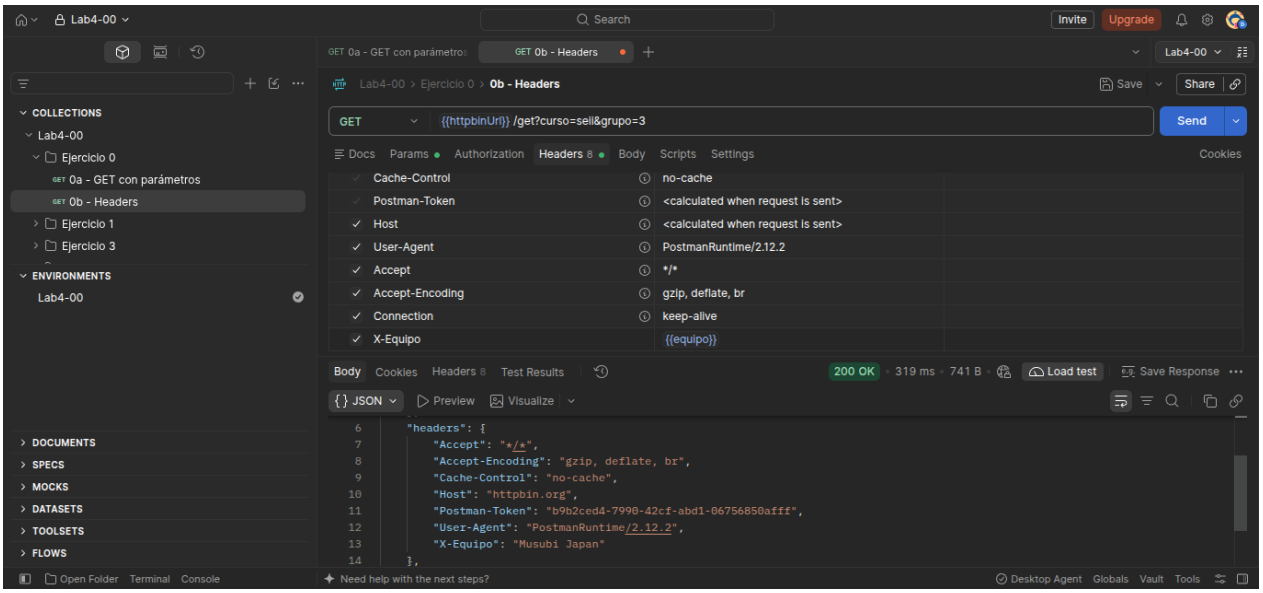


Figura 2. La misma petición con el header `X-Equipo`.

(c) Cuerpo JSON

El cuerpo enviado a `POST https://httpbin.org/post` es:

```
{
  "equipo": "Musubi Japan",
  "grupo": 0,
  "integrantes": ["Daniel Alonso Gracia Pinto"],
  "proyecto": "App móvil para aprender japonés"
}
```

El header que le dice al servidor que el cuerpo es JSON es `Content-Type: application/json`. Postman lo pone solo al elegir *raw* y *JSON*. La respuesta trae el cuerpo dos veces: en `data`, como texto, y en `json`, ya convertido en objeto.

Al cambiar a *Text*, Postman envía `Content-Type: text/plain`. El httpbin.org de hoy sigue llenando `json` con ese header: intenta leer el cuerpo como JSON sin mirar el `Content-Type`. Lo comprobé con un cuerpo que no es JSON (`hola equipo`): ahí `json` sale en `null` y el texto solo queda en `data`. El httpbin original, en cambio, devolvía `json: null` siempre que el tipo no fuera `application/json`. Eso es lo que debería hacer una API real: no adivinar el formato, sino responder `415 Unsupported Media Type` si el `Content-Type` no es el que espera.

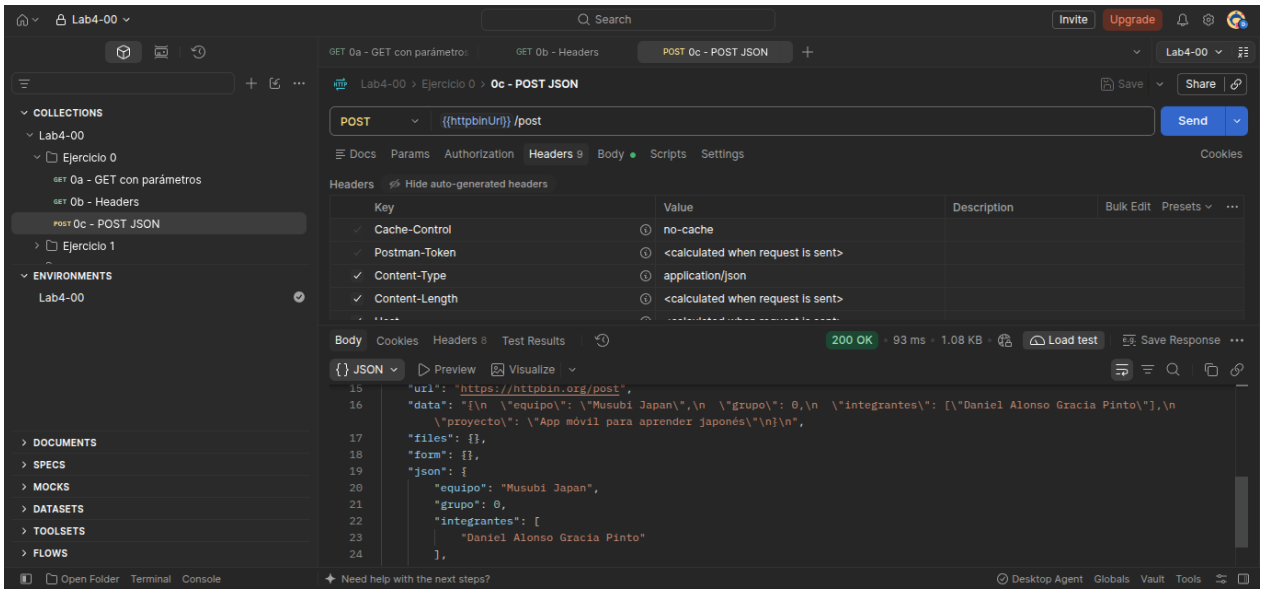


Figura 3. `POST /post` con el cuerpo como JSON.

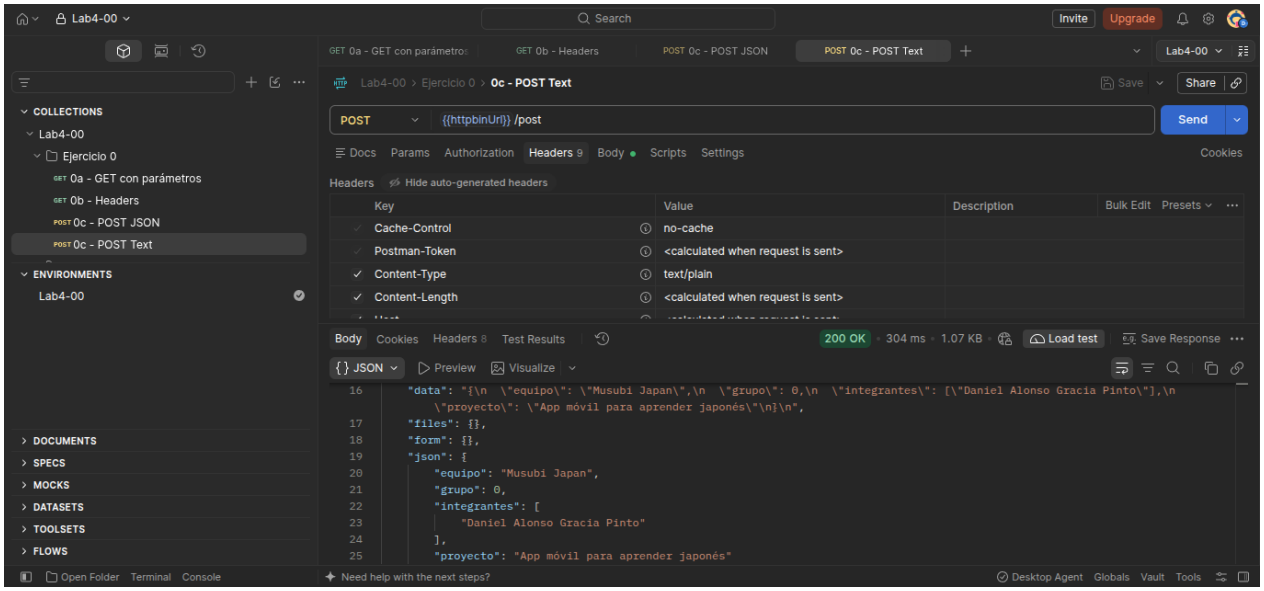


Figura 4. El mismo cuerpo enviado como *Text*: `json` sigue lleno.

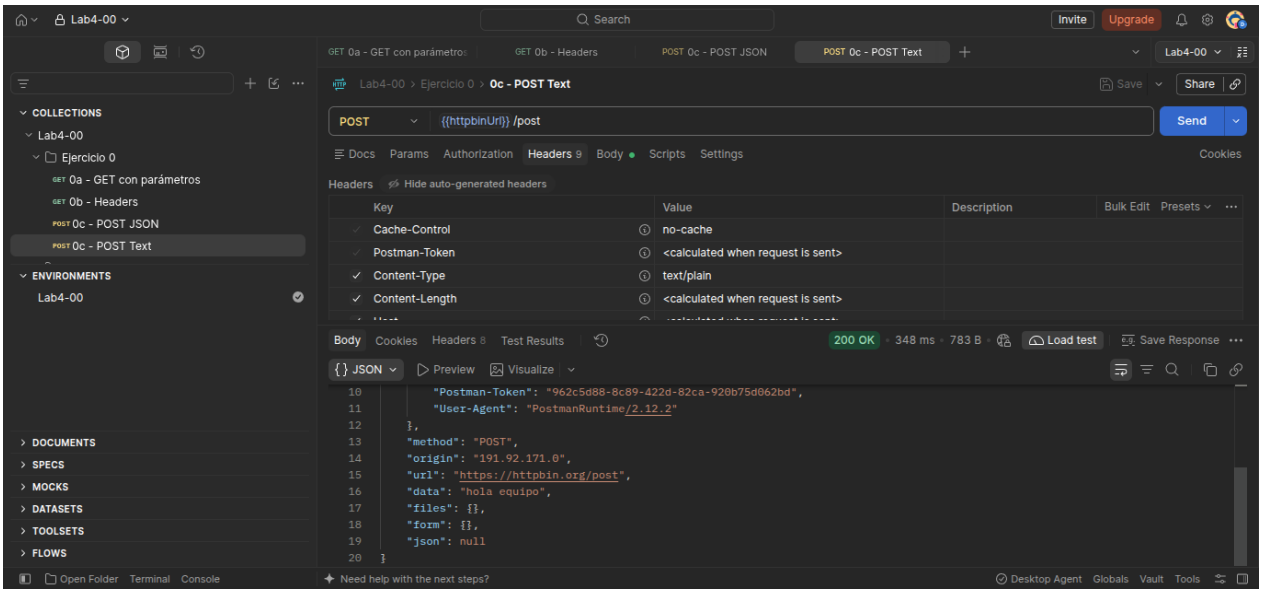


Figura 5. Un cuerpo *Text* que no es JSON: `json` sale en `null`.

(d) Códigos de estado

Las tres respuestas llegan con el cuerpo vacío (`Content-Length: 0`). La app solo cuenta con el código para decidir qué mostrar.

Código	Familia	Responsable	Qué muestra la app
201 Created	2xx	No hay error: el servidor creó el recurso.	Una confirmación (“Préstamo registrado”) y el recurso nuevo.
404 Not Found	4xx	El cliente: pidió un recurso que no existe o una URL mal armada.	“No encontramos lo que buscas” y la opción de volver. Reintentar no sirve.
500 Internal Server Error	5xx	El servidor: falló al procesar una petición válida.	“Algo salió mal de nuestro lado, intenta más tarde”, con un botón para reintentar y sin detalles técnicos.
429 Too Many Requests	4xx	El cliente: superó el límite de peticiones del servidor.	Nada de inmediato: espera lo que indique el servidor y reintenta sola. Si tarda, “Hay mucho tráfico, reintentando…”.

El 429 no se pidió: apareció al enviar la petición de (a). httpbin.org admite 17 peticiones por segundo por IP. Postman estaba usando su *Cloud Agent*, que envía la petición desde los servidores de Postman, con una IP que comparten muchos usuarios a la vez; la misma petición enviada con `curl` desde el equipo respondió 200. Al pasar al agente de escritorio, la petición salió con la IP propia y el 429 desapareció. A diferencia de los códigos de `/status`, este trae cuerpo, con el motivo y el tiempo de espera: `retry after 1 second`. Reintentar sí sirve aquí, pero esperando; reintentar enseguida solo alarga el bloqueo.

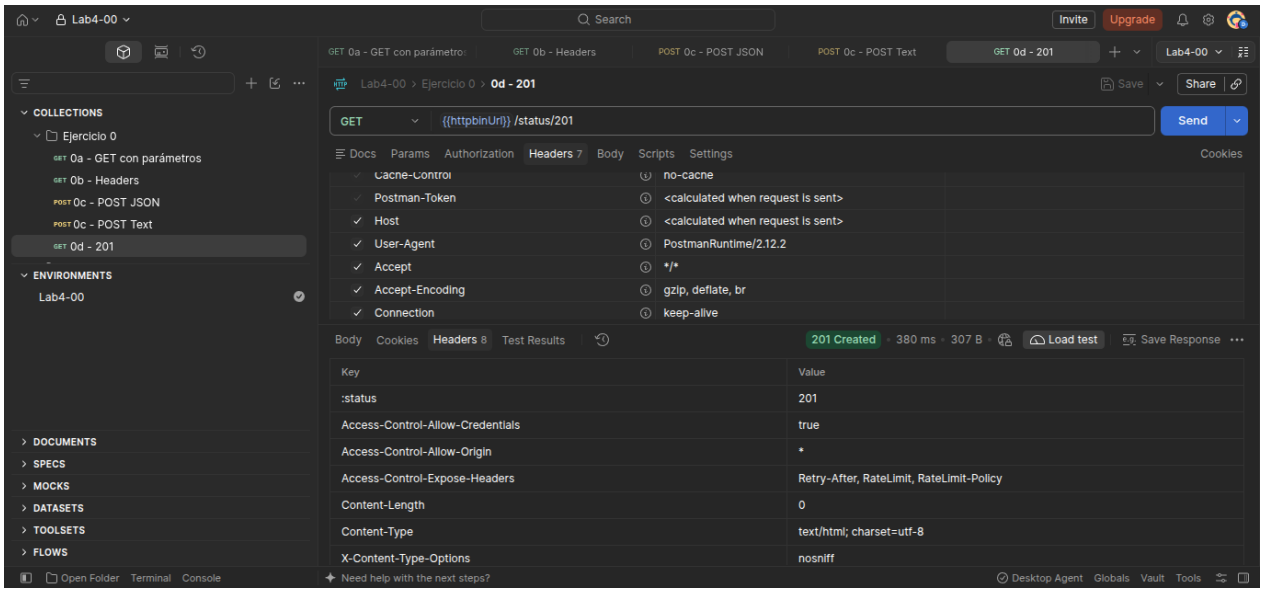


Figura 6. Respuesta de `/status/201`.

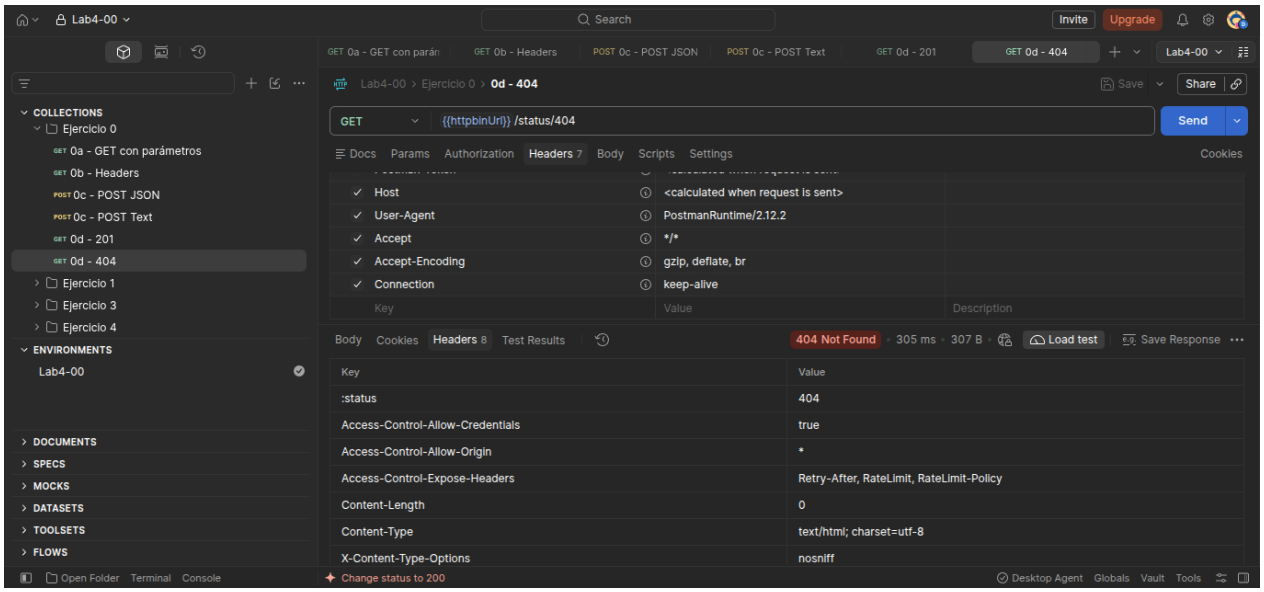


Figura 7. Respuesta de `/status/404`.

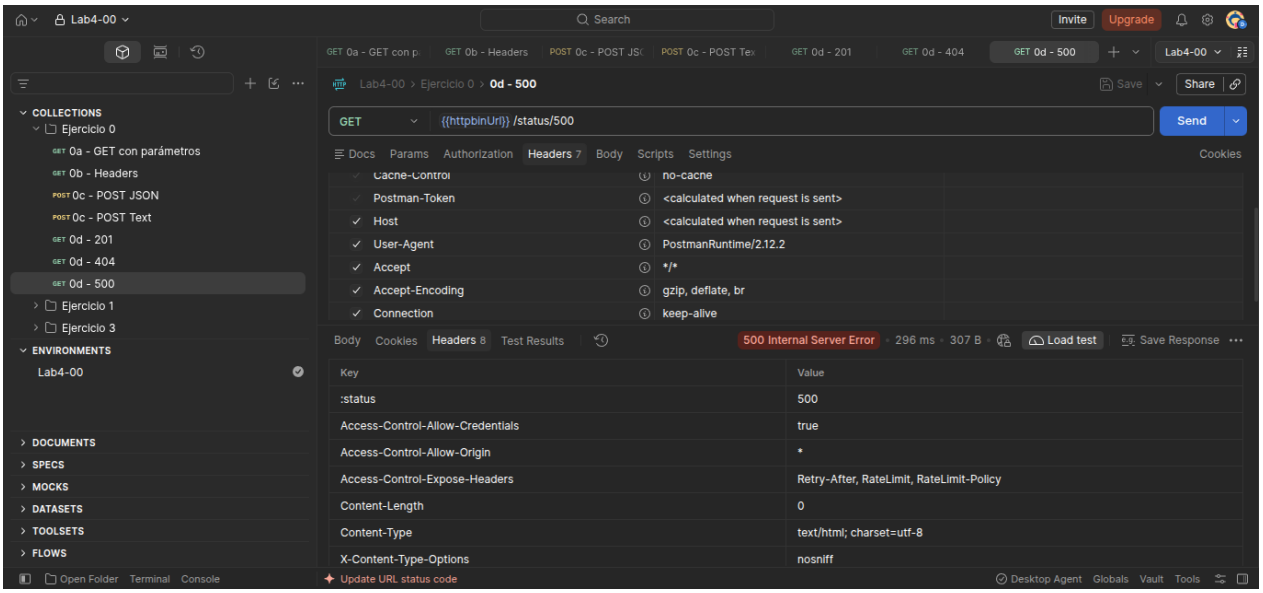


Figura 8. Respuesta de `/status/500`.

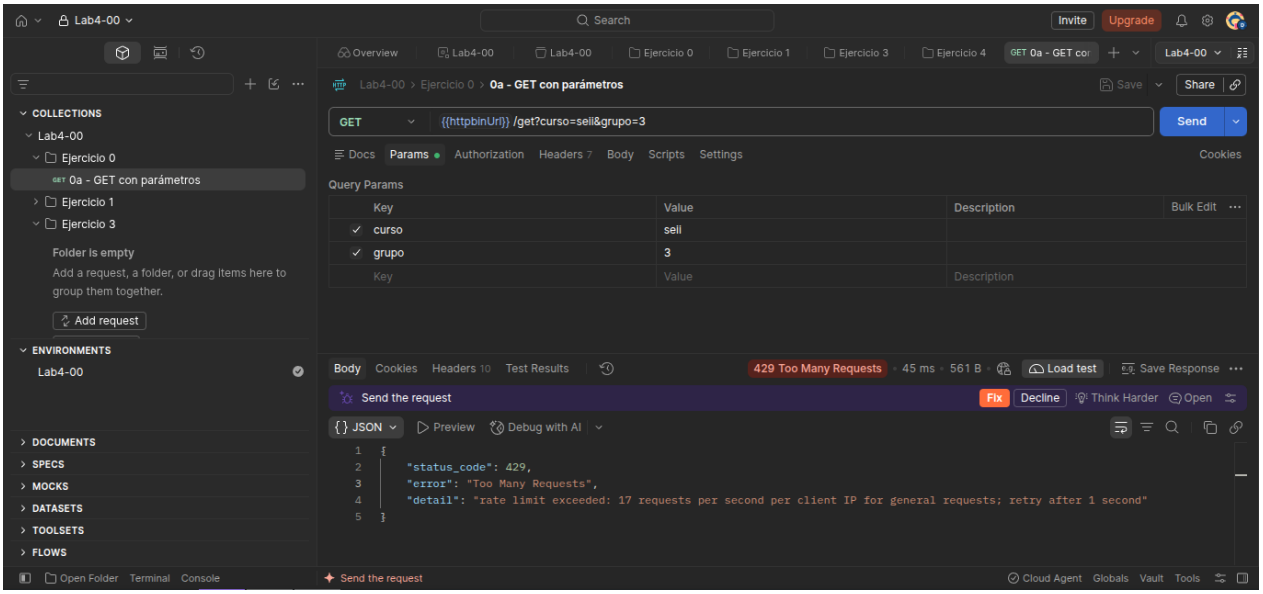


Figura 9. El 429 que devolvió httpbin.org al superar el límite de peticiones.

Ejercicio 1. CRUD completo con JSONPlaceholder

Ejercicio 2. Un cliente REST en Java

Ejercicio 3. Los seis protocolos en la práctica

Ejercicio 4. Diseño y documentación de una API REST